

Construção e avaliação do modelo Tensorflow

Olá!

Neste vídeo, iremos aprender a construir o modelo e como avaliá-lo, de acordo com a métrica de acurácia uma vez que estamos trabalhando com uma classificação.

Inicialmente, já tendo importado as bibliotecas Keras Tensorflow e NumPye nós iremos executar o código a seguir.

Mas iremos, primeiramente, definir a arquitetura do modelo.

Iremos colocar o comando "model", que seria uma variável para guardar o modelo. Então, fica: `model = keras.Sequential([`. Iremos passar uma lista e os seguintes comandos:

```
keras.layers.Flatten(input_shape=(28, 28)),
```

Até aí, nós estamos construindo um modelo que é sequencial, ou seja, cujas camadas são uma em sequência da outra o resultado de uma vai para a próxima.

Nesta primeira camada que instruímos, estamos utilizando uma "layer" que é camada, o Keras, de "Flatten" então, estamos pegando a matriz, que tem tamanho 28 x 28 e a traduzindo em um vetor de 28 vezes 28 porque esse modelo de rede neural a qual estamos utilizando no momento não é capaz de trabalhar com matrizes.

Logo, o objetivo será transformar a matriz em um vetor simplesmente concatenando as linhas dessa matriz.

Essa função "Flatten" já faz isso por nós e nós passamos o tamanho do vetor de entrada com o parâmetro: `(input_shape=(28, 28))`.

Após essa primeira linha, nós iremos colocar:

```
keras.layers.Dense(128, activation=tf.nn.relu)
```

```
keras.layers.Dense(10, activation=tf.nn.softmax)
```

```
])
```

Nessas duas últimas camadas que foram ditas aqui aplicamos duas camadas densas, as camadas comuns de redes neurais nas quais todos os neurônios da camada anterior se ligam aos neurônios da camada atual e utilizamos uma função de ativação para cada uma dessas camadas onde, na primeira, estamos utilizando a "ReLU" uma função com a qual qualquer número negativo se torna 0 e se o número for positivo, mantém-se o número, ou seja, essa função evita números negativos.

E, por fim, uma última camada densa, de 10 neurônios com a ativação "softmax", que é uma função boa para classificação.

Se formos perceber, estamos usando, na última camada densa, 10 neurônios.

É a última camada, uma camada de saída.

Se notarmos, nós temos 10 classes no conjunto de dados.

Então, essa última camada bate, exatamente, no seu tamanho com a quantidade de classes.

Devemos, sempre, tomar esse cuidado: a última camada da rede neural deve bater com o número de classes na saída e a função "softmax" faz uma computação de

maneira que nós teremos essas classes bem definidas para podemos utilizar uma avaliação em seguida.

Após a construção desse modelo, podemos utilizar o seguinte comando `"model.summary()"`.

Esse comando nos dirá quantos pesos nós temos na rede neural e a estrutura da mesma.

Ao executar esses dois comandos, a definição do modelo e o sumário nós teremos um resumo, de fato, do modelo.

O modelo é do tipo sequencial, e vai possuir três camadas: a primeira é a camada "Flatten", cuja saída será "784" onde 784 é a multiplicação de 28 por 28 e não temos nenhum parâmetro de peso afinal, estamos apenas convertendo uma matriz em um vetor.

A segunda camada é a camada "Dense" onde a saída será "128" que é exatamente igual ao número de neurônios nesta camada.

E o número de parâmetros, ou seja, de pesos que teremos nesta camada é igual a 100480 que é exatamente os 784 elementos de entrada dessa camada vezes os 128 neurônios mais 128.

O que é isso?

Nós aprendemos na aula sobre redes neurais que para cada elemento de entrada nós temos um peso para cada neurônio.

Por isso, a multiplicação de 784 vezes 128.

E cada neurônio possui um fator de viés, ou bias.

Então, este bias será somado também, pois também é um peso e para cada um dos neurônios, nós teremos um bias.

Dessa maneira, 784 vezes 128 mais 128 é exatamente igual a 100480 parâmetros.

Na camada seguinte, a próxima camada "Dense" é composta de 10 neurônios, ou seja, a saída dessa camada será igual a 10 um neurônio para cada classe e teremos 1290 parâmetros nessa classe, ou seja, 128 de entrada vezes 10 neurônios: 1280 mais os 10 bias, um para cada neurônio: 1290.

No total, teremos 101770 parâmetros e todos eles serão treináveis, ou seja, todos serão apreendidos no passo de treinamento da rede neural.

A seguir, nós iremos compilar esse modelo utilizando alguns parâmetros no seguinte comando: `"model.compile(")`. Deve ficar assim:

```
model.compile(optimizer = 'adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

Os três parâmetros usados foram: `"optimizer"`, `"loss"` e `"metrics"`.

O `"adam"` é um otimizador para os pesos. Ele vai ser a técnica utilizada para a construção dos pesos para o treinamento.

O `"loss"` é a métrica de erro que será a entropia cruzada categórica esparsa.

Com esse nome mais complicado, a entropia cruzada categórica esparsa é justamente uma técnica para tentar trabalhar com um resultado que possui muito valores próximos a 0 e alguns valores próximos a 1 que é exatamente o que iremos usar nos neurônios de saída.

Um neurônio correspondente a uma certa classe terá maior valor em relação aos demais.

E, por fim, iremos utilizar como métrica a "accuracy", ou seja, a porcentagem de elementos classificados corretamente sobre o total.

Assim, podemos compilar o modelo e, por fim, iremos treiná-lo, utilizando: `model.fit(train_images, train_labels, epochs=5)`.

Iremos utilizar cinco épocas de treinamento, ou seja, cinco iterações dos conjuntos de dados para tentar construir um bom modelo.

Esta quantidade de épocas pode ser aumentada ou diminuída de acordo com o modelo, com a dificuldade dos dados.

Dados mais complexos merecem mais épocas.

Dados menos complexos, menos épocas.

Para este conjunto de dados, só cinco épocas já bastam.

Executando o comando, teremos uma representação gráfica das épocas sendo treinadas, o número de elementos sendo treinados o valor de erro e a acurácia.

Na primeira época, a acurácia conquistada foi de 82%. Na segunda, 86,36%. Na terceira, 87,89%. Na quarta, 86,66%. E na quinta, 89,04%.

Dessa maneira, o modelo tem acurácia de 89,04% para os dados de "train".

Vale ressaltar que esta acurácia é para os dados de "train" e essa quantidade de épocas foi escolhida pelo critério que podemos perceber na evolução das épocas: da primeira para a segunda época, tivemos um aumento de 4% de acurácia.

Da segunda para a terceira, já podemos perceber que foi apenas 1,5% de melhora.

Na terceira, podemos perceber 1,4%.

E na quarta, já foi menos que 1%. Ou seja, mais épocas poderiam ocasionar um overfitting.

Poderíamos ter uma métrica muito boa para treinamento, mas métricas não tão boas para teste.

Por fim, podemos avaliar o conjunto de dados sobre os dados de "test".

Para isso, usaremos o seguinte comando: `test_loss, test_acc = model.evaluate(test_images, test_labels)`. "acc" é referente a accuracy.

Com isso, o modelo irá avaliar as imagens de "test" e seus rótulos e irá guardar tanto o valor de erro na variável "test_loss" quanto o valor de acurácia, na variável "test_acc".

Logo em seguida, podemos usar: `print('Test accuracy:', test_acc)`.

Com isso, iremos printar a acurácia do "test" e a acurácia do conjunto "test" foi de 88,05% uma acurácia relativamente elevada para o conjunto de dados.

Com isso, conseguimos treinar o modelo defini-lo e avaliá-lo de uma maneira simples.

Se quisermos avaliar individualmente, podemos utilizar o método "predict" que irá criar uma predição para cada uma das imagens de "test".

Iremos utilizá-lo agora.

Então, podemos criar uma variável chamada "predictions", ficando: `predictions = model.predict(test_images)`.

Com isso, iremos criar uma predição para cada uma das imagens de "test".

Imprimindo a primeira predição, nós iremos ter o seguinte comando: `predictions[0]`, ou seja, o índice "0" de "predictions".

Nós teremos um array, com 10 elementos cada um deles números em notação científica todos eles bem próximos a 0, são valores bem pequenos com exceção de um deles.

A classificação se dará pelo maior valor de predição que teremos dentre esses "predictions".

Ao invés de ter que olhar e verificar cada um desses valores nós podemos utilizar o NumPye para verificar o índice cujo elemento é o máximo. Dessa maneira, nós saberemos qual foi a predição da rede neural sem precisar verificar elemento a elemento.

Então, iremos utilizar: `np.argmax(predictions[0])`.

Ao executar isso, iremos perceber que foi o elemento de índice "9" que é o maior de todos, ou seja, temos a classe "9".

A classe "9" é a que corresponde à classificação.

Se formos comparar o rótulo da imagem 0 teremos que o rótulo dela é "9", ou seja, para a primeira imagem de "test", o modelo acertou a predição.

Se quisermos fazer uma avaliação separada e individual de cada uma das imagens, podemos utilizar imagem por imagem.

Iremos colocar a primeira imagem de "test" em uma variável separada para facilitar o entendimento. Então, iremos efetuar a seguinte linha de comando: `img = test_images[0]`. Se formos printar o "shape" dessa imagem, desse "img" teremos "28, 28".

Ou seja, a célula anterior foi:

```
img = test_images[0]
```

```
print(img.shape)
```

Teremos "28, 28", ou seja, a primeira imagem do conjunto de "test" tem 28 x 28.

Porém, o modelo espera que um vetor de imagens seja passado e aqui, nós temos apenas uma imagem sozinha.

Então, iremos utilizar mais uma vez o NumPye para expandir a dimensão dessa imagem e ter um vetor como única imagem.

Iremos usar o seguinte comando: `img = (np.expand_dims(img,0))`.

"expand_dims" seria expand dimension.

Na linha seguinte, iremos colocar: `print(img.shape)`.

Esse "expand_dims" vai aumentar as dimensões da imagem colocando-a dentro de um novo vetor.

Então, ao executar este comando, nós teremos, agora o "shape" dessa imagem como "1, 28, 28", ou seja, é um vetor unitário com uma imagem de 28 x 28.

Esta nova imagem, com dimensões expandidas pode ser aplicada ao modelo, já que ele espera um vetor de imagens e teremos "predictions_single" podemos colocar uma variável, chamada "predictions_single" somente para armazenar a predição de uma única imagem. Fica assim: `predictions_single = model.predict(img)`. Em seguida, iremos usar: `print(predictions_single)`.

Com isso, teremos a predição da única imagem que serão aqueles números bem próximos a 0 ou a 1 em notação científica.

Mais uma vez podemos utilizar o NumPye "argmax" para efetuar as predições, então: `np.argmax(predictions_single[0])`, que irá nos retornar, justamente, o rótulo da primeira imagem.

Então, com isso, estamos encerrando a aula de Tensorflow tendo mostrado como construir um modelo e como avaliá-lo.

Espero que a aula tenha sido proveitosa para todos.

Até a próxima!